



Box model

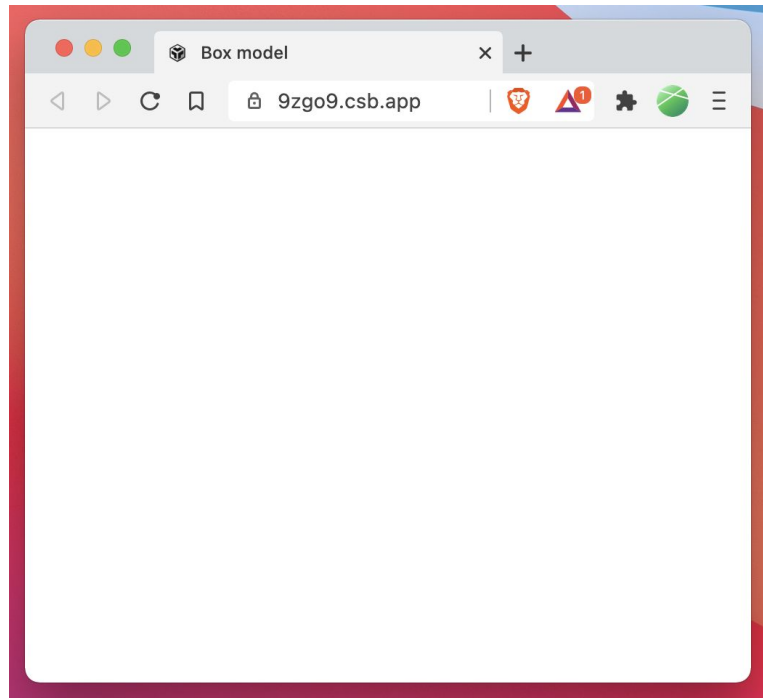


Box model

Take a look at this snippet:

```
<body>  
  <h1>Hello!</h1>  
</body>
```

Q: What will this page look like?



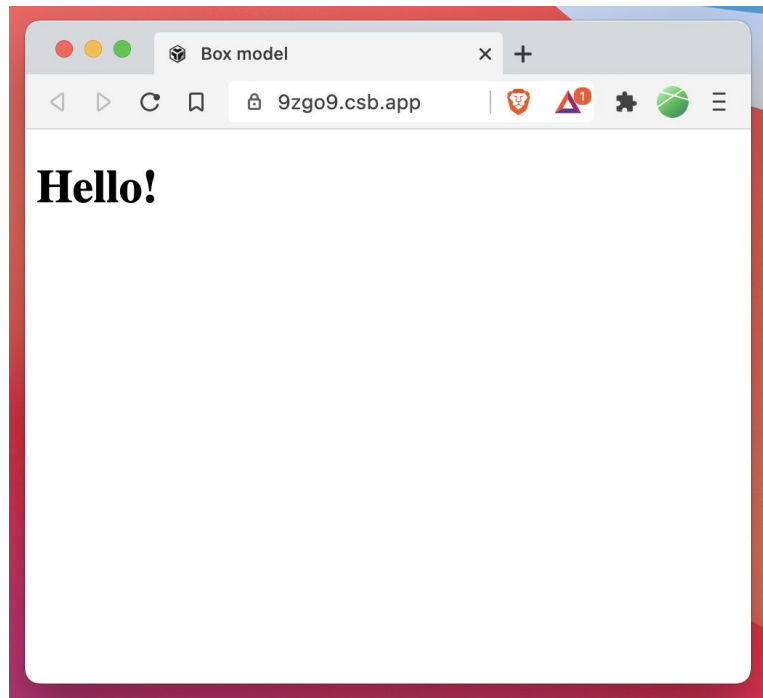


Box model

Nice!

```
<body>  
  <h1>Hello!</h1>  
  <span>Hi!</span>  
</body>
```

Q: And now?





Box model

Awesome!

```
<body>
```

```
  <h1>Hello!</h1>
```

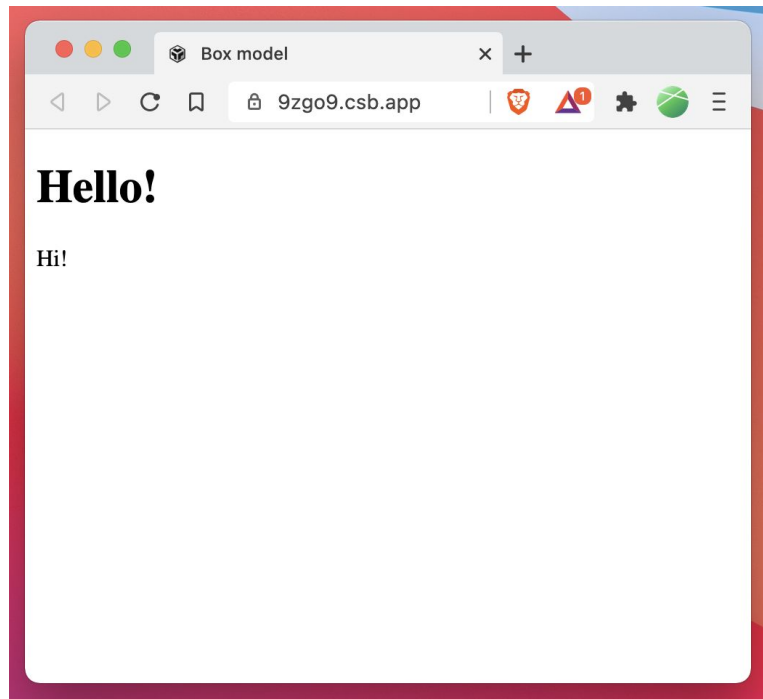
```
  <span>Hi!</span>
```

```
  <span>Hi!</span>
```

```
  <span>Hi!</span>
```

```
</body>
```

Q: How about now?





Box model

🤔 Was that what you expected?

```
<body>
```

```
  <h1>Hello!</h1>
```

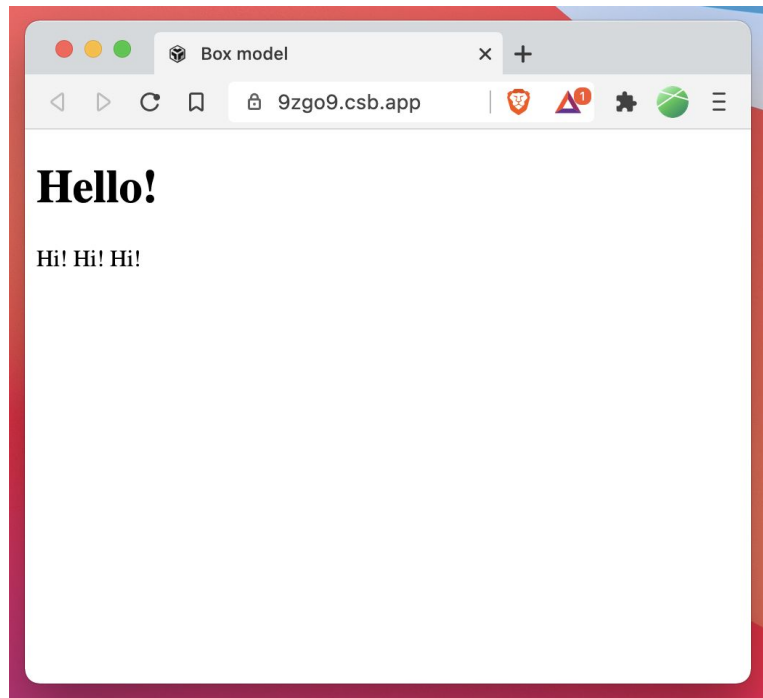
```
  <span>Hi !</span>
```

```
  <span>Hi !</span>
```

```
  <span>Hi !</span>
```

```
</body>
```

Q: What do you think is going on?



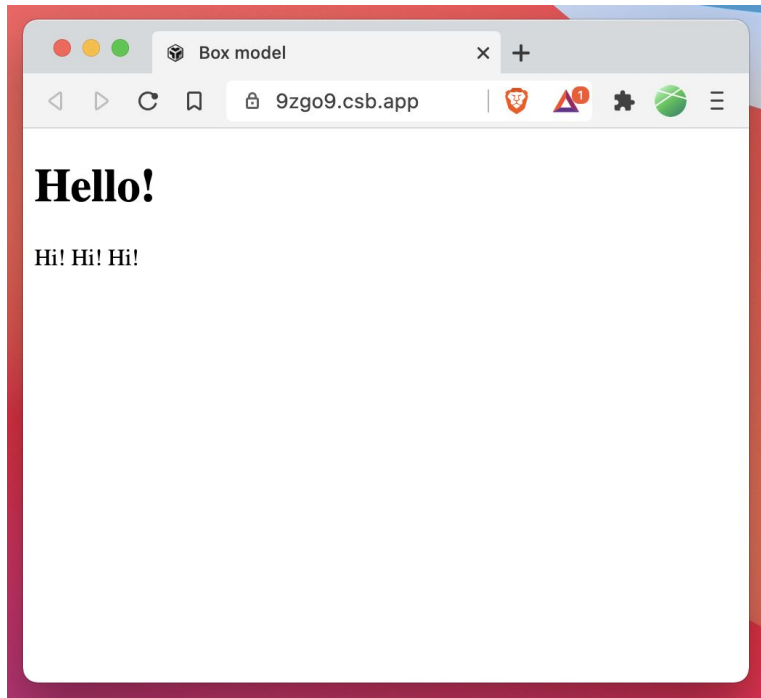


Box model

Some elements seem to be occupying the **entire width** of the screen and kicking the following element down to the next line

Others appear quite happy to live cozily **side by side**

Let's take a look at the browser to see what's up!





Block vs inline

Every element on the page can be thought of as a **box**

There are **2 kinds** of boxes in HTML: **block boxes** and **inline boxes**

We can **change** a box's **type** by changing its CSS **display property**

Q: Why do you think we need these 2 different behaviours?



Time for some live coding

Remember our book?

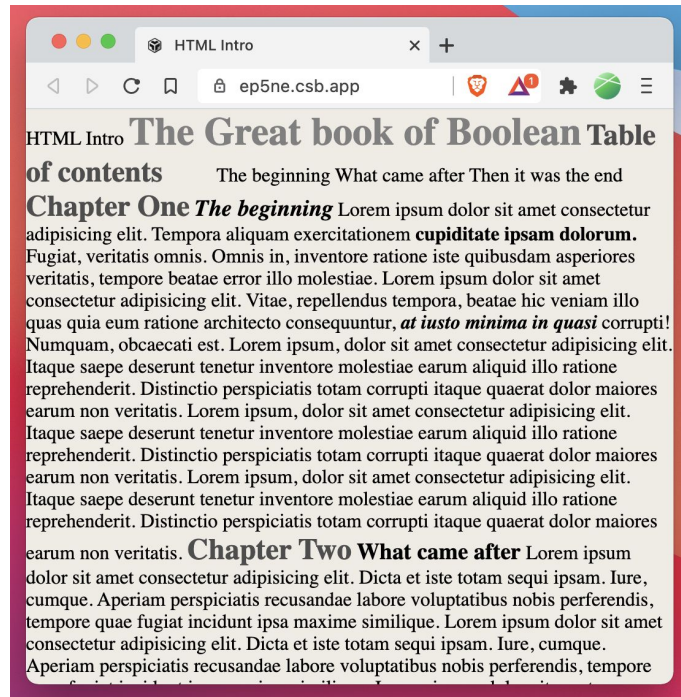
Let's take a look at it and see why blocks and inline elements matter!



Block vs inline

Hopefully you can see why it was necessary to make this distinction

Since everything is a box, if everything was **inline**, things like **titles** would not be able to take up the **whole width** of the page so easily, and everything would get **squished**, like a typical London commute

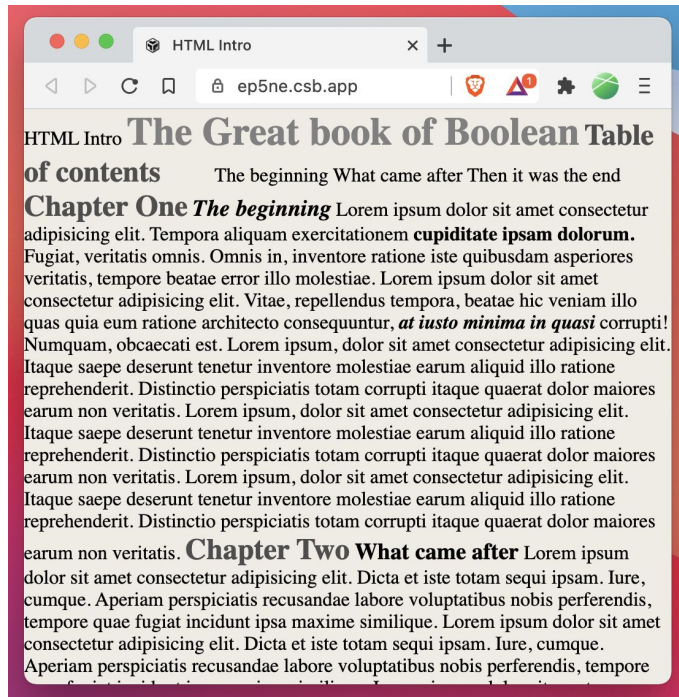




Block vs inline

Note that inline elements get squished, so their **width** and **height** is **not respected**

There's a hybrid display type, **inline-block**, that allows an element to be next to others, and have a width and height

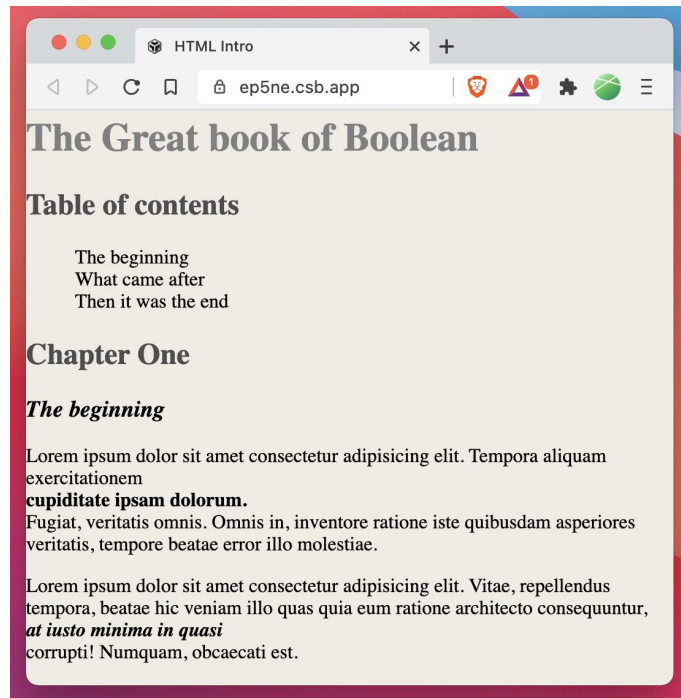




Block vs inline

On the other hand, if everything was a **block**, we would not be able to **nest elements** that are supposed to be together, like `<strong>` and `<em>` inside `<p>`s, since they would break the paragraph

No two elements could ever be **together**, like a tragic love story





What makes up a box

On top of inline and outline, each box is made up of **four layers** that can **take up space** on the page, these are:

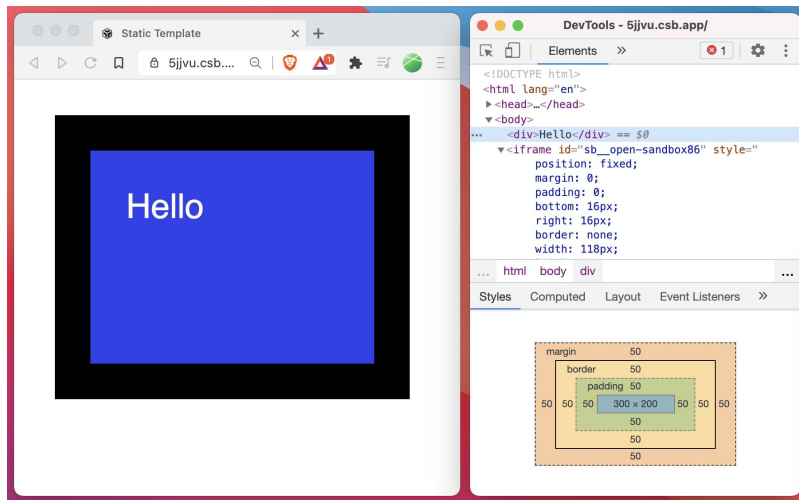
- The box itself
- Padding
- Border
- Margin

Let's take a closer look at them using our DevTools!



What makes up a box

```
div {  
  width: 300px;  
  height: 200px;  
  
  padding: 50px;  
  border: solid 50px black;  
  margin: 50px;  
}
```



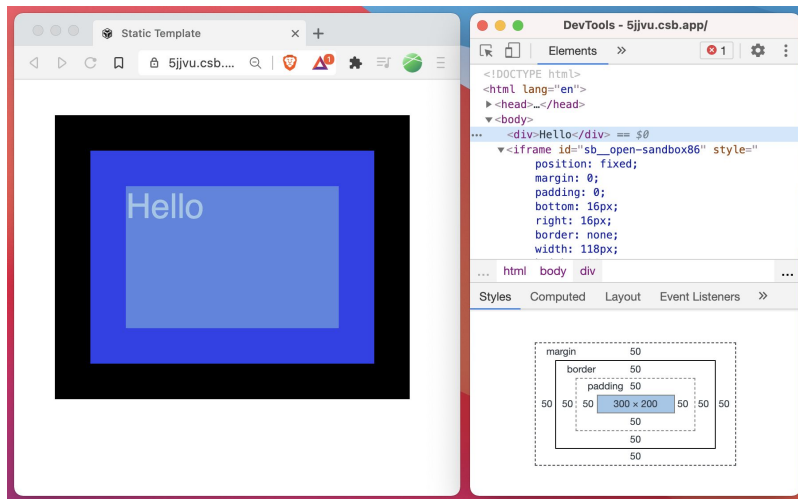
Here you can see them all in action

But let's take a look at them one at a time to see what's going on



Size

```
div {  
  width: 300px;  
  height: 200px;  
  
  padding: 50px;  
  border: solid 50px black;  
  margin: 50px;  
}
```



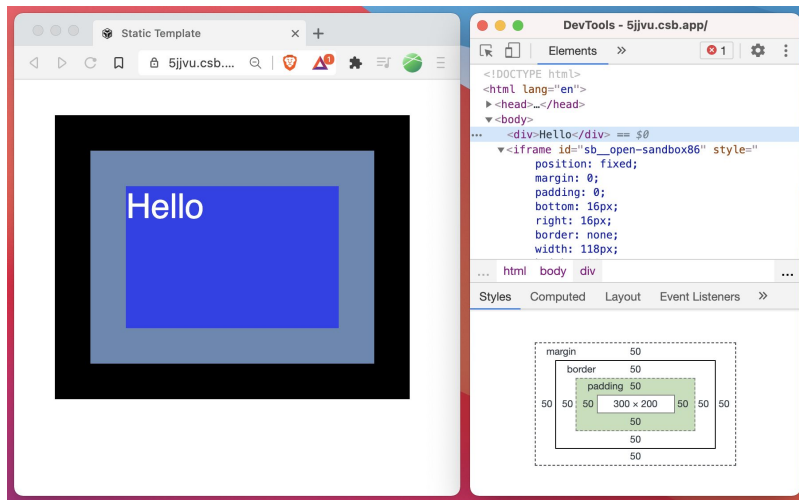
Starting from within, we have the **size of the box itself**

In this case we've set it to **300px** by **200px**



Padding

```
div {  
  width: 300px;  
  height: 200px;  
  
  padding: 50px;  
  border: solid 50px black;  
  margin: 50px;  
}
```



Next there's **padding**

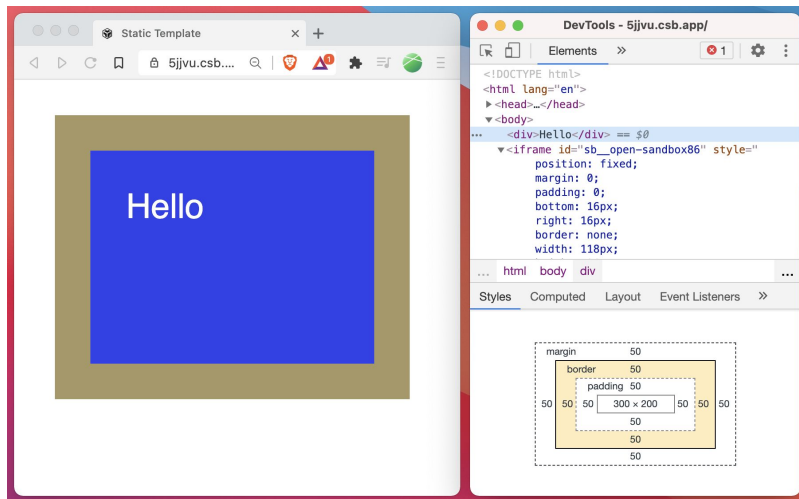
This is **extra spacing** we can add **within the box**, to give the content some breathing room

We can add it to **all sides equally**, or tweak the padding on **each side individually**, more [here](#)



Border

```
div {  
  width: 300px;  
  height: 200px;  
  
  padding: 50px;  
  border: solid 50px black;  
  margin: 50px;  
}
```



Then we have the **border**

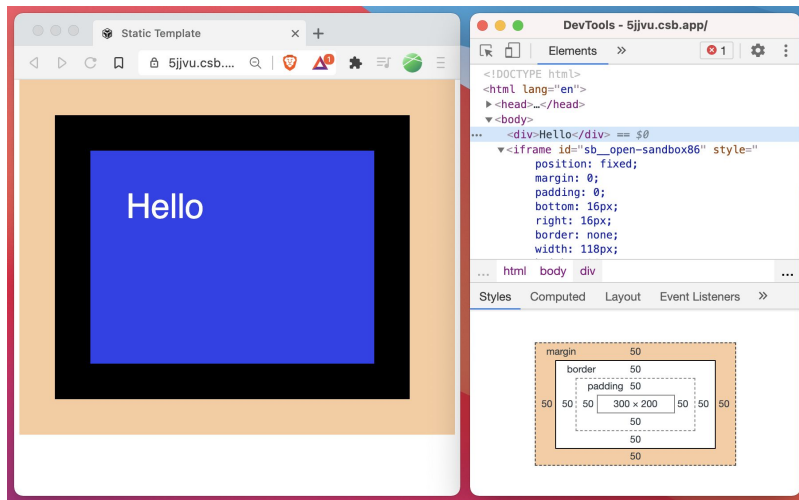
As the name suggests this is a **line** that marks the **end of the box**

Like padding, we could tweak **each side individually**, more [here](#)



Margin

```
div {  
  width: 300px;  
  height: 200px;  
  
  padding: 50px;  
  border: solid 50px black;  
  margin: 50px;  
}
```



Finally, there's the **margin**

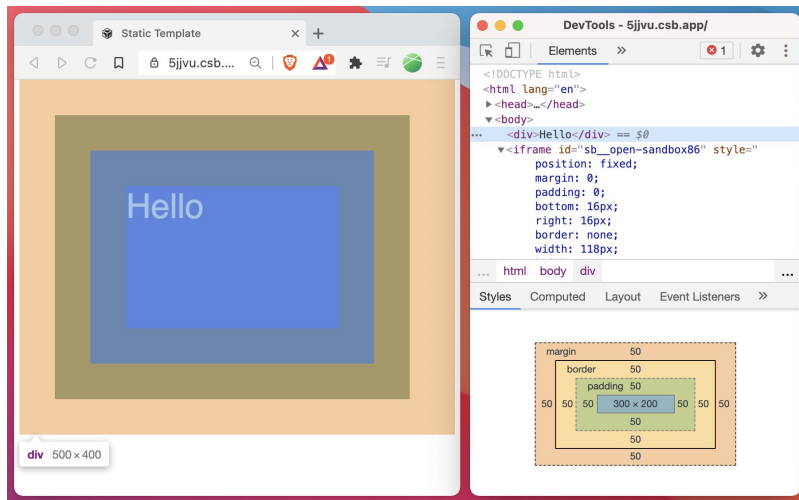
This is the **space** we want to add **outside of the box**

Once again, we can tweak **each side individually** if we need to, you can read more [here](#)



Space on the page

```
div {  
  width: 300px;  
  height: 200px;  
  
  padding: 50px;  
  border: solid 50px black;  
  margin: 50px;  
}
```



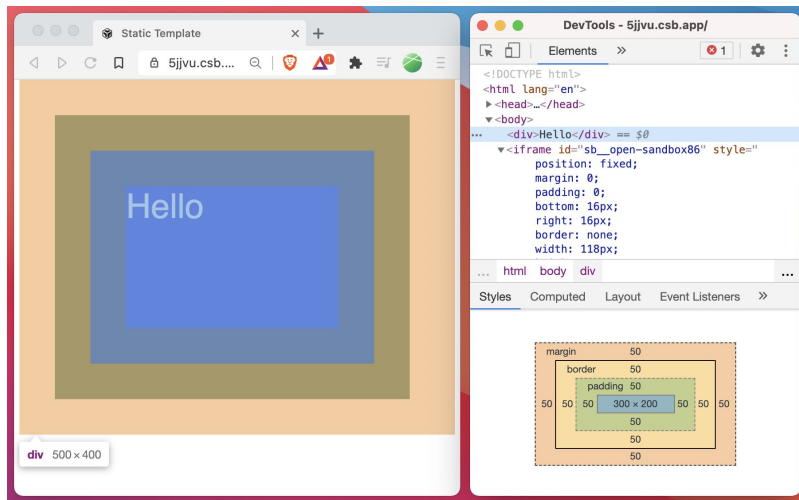
In this example, you can see how, even though the element itself is **300px** by **200px**, after we add the **padding**, **border** and **margin**, we get to **500px** by **400px**, which is the **actual size** this element is taking up on the page!

Q: Is that calculation correct? What's going on?



Space on the page

```
div {  
  width: 300px;  
  height: 200px;  
  
  padding: 50px;  
  border: solid 50px black;  
  margin: 50px;  
}
```

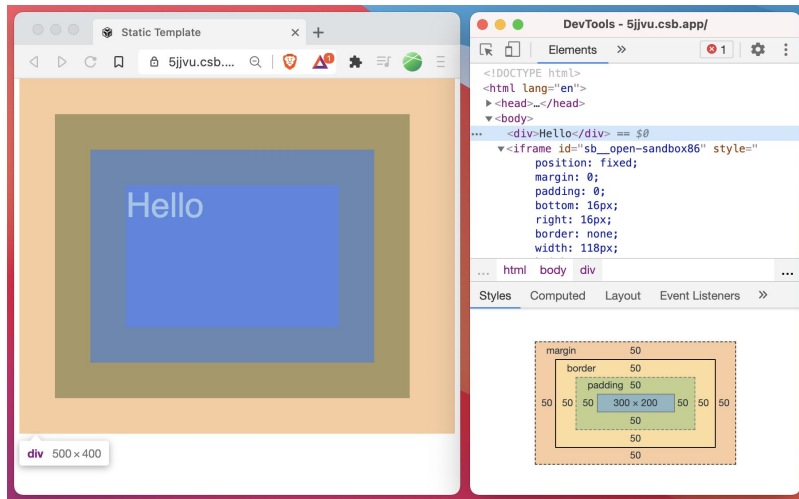


A: Even though the element is taking up **600px** by **500px**, the margin is not being taken into account by the inspector!



Space on the page

```
.a-box {  
  width: 300px;  
  height: 200px;  
  
  padding: 50px;  
  border: solid 50px black;  
  margin: 50px;  
}
```



Using the **div** selector will change **every div on the page**

We can be more **specific** by giving our element the attribute **class="a-box"**

CSS allows us to select elements by class by prefixing the selector with a **dot** .

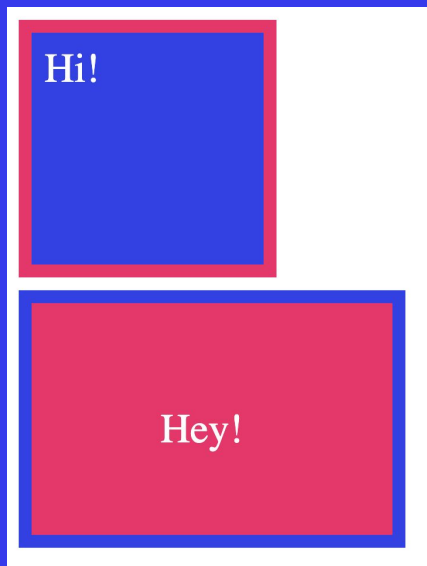


Time for some live coding

Let's make our book have proper pages!



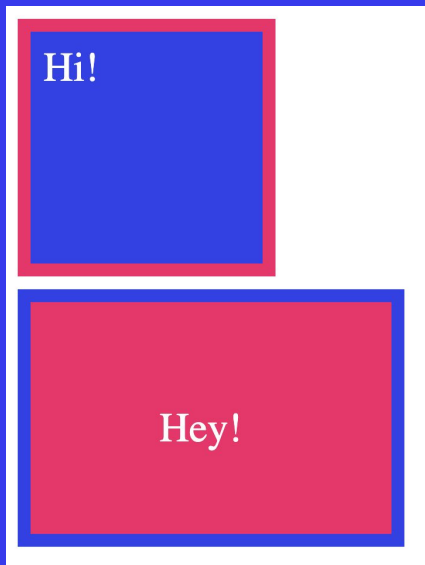
Time for some practice



Try to replicate this page as best as you can!



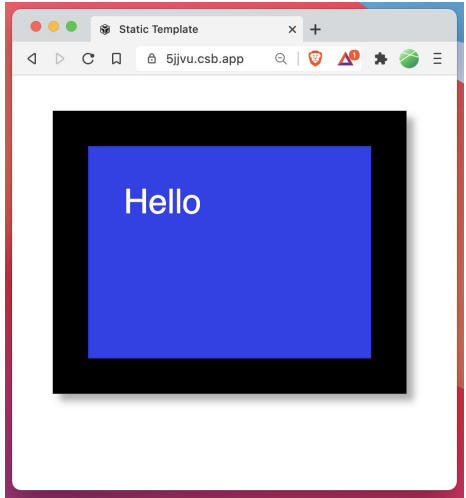
Time for more practice!



Good job!
Now, try getting it pixel perfect by inspecting the page!



Box shadow



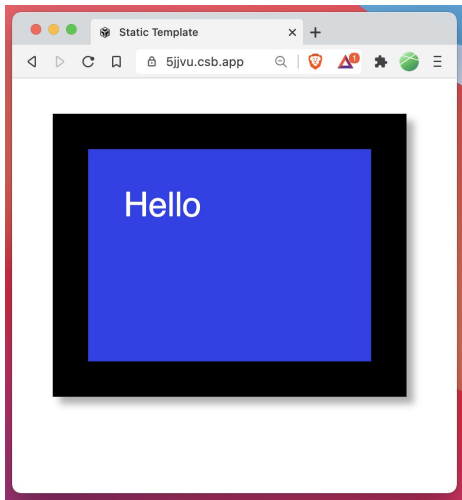
```
.a-box {  
  ...  
  box-shadow: 10px 10px 10px 0px #00000050;  
}
```

Another cool property we can add to our blocks is **box-shadow**

You don't want to overdo this one, but in the right places it can add a nice touch!



Box shadow

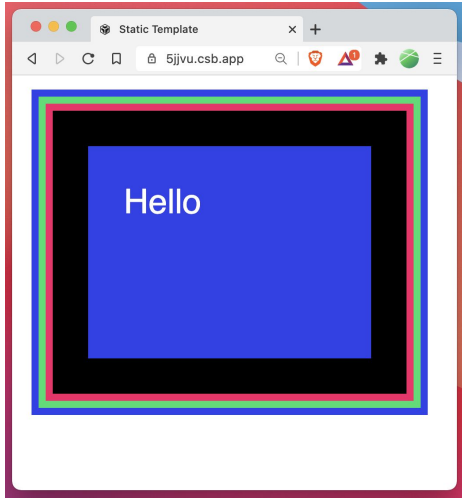


```
.a-box {  
  ...  
  box-shadow: 10px 10px 10px 0px #00000050;  
}
```

From left to right, the values represent: **x-offset**, **y-offset**, **blur-radius**, **spread-radius** and **color**



Box shadow



```
.a-box {  
  ...  
  box-shadow:  
    0 0 0 10px #f71568, /* shadow one */  
    0 0 0 20px #00de6b, /* shadow two */  
    0 0 0 30px #363aeb; /* shadow three */  
}
```

An interesting thing about box shadow is that a single element can have **as many as you want!**

Beware of the **syntax** though, each shadow is separated by a **comma**, with a **semicolon** after the last one



Time for some live coding

Let's add shadows to our book pages!



The great reset

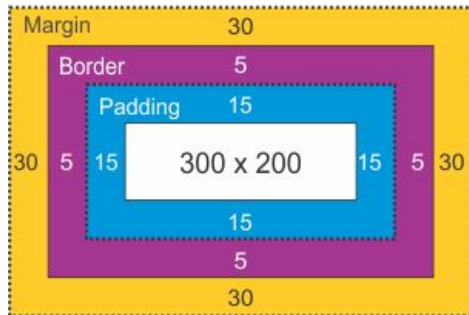
```
* {  
  margin: 0;  
  padding: 0;  
  box-sizing: border-box;  
}
```

Q: What do you think this is doing?



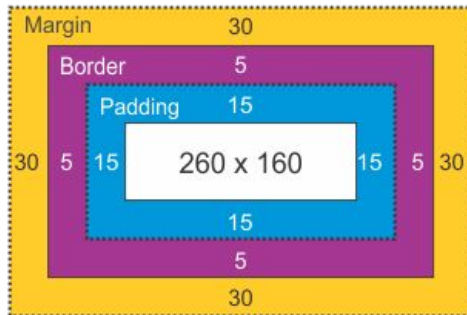
Box-sizing

340x240



```
div {  
  width: 300px;  
  height: 200px;  
  padding: 15px;  
  border: 5px solid black;  
  margin: 30px;  
  box-sizing: content-box;  
}
```

300x200

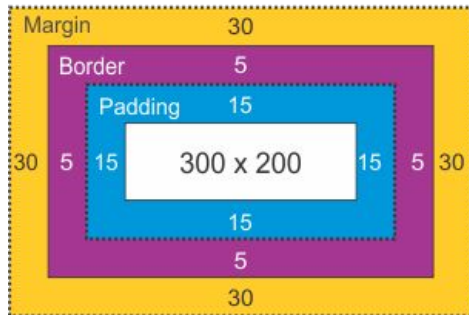


```
div {  
  width: 300px;  
  height: 200px;  
  padding: 15px;  
  border: 5px solid black;  
  margin: 30px;  
  box-sizing: border-box;  
}
```

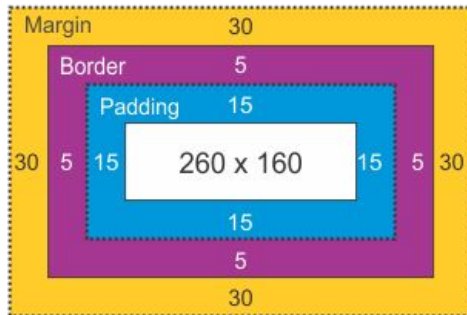
Q: What's different between these two examples?



Box-sizing



```
div {  
  width: 300px;  
  height: 200px;  
  padding: 15px;  
  border: 5px solid black;  
  margin: 30px;  
  box-sizing: content-box;  
}
```



```
div {  
  width: 300px;  
  height: 200px;  
  padding: 15px;  
  border: 5px solid black;  
  margin: 30px;  
  box-sizing: border-box;  
}
```

border-box makes it so that the size of an element will **include** its **padding** and **border** sizes



Normalising

What we used earlier was a simple way to **reset** and adjust some useful properties

There are also more involved CSS resets that aim to **standardise** how CSS is applied across browsers

A great example is [normalize.css](#)



CSS selectors

So far we've seen **tag selectors**:

```
p {  
  . . .  
}
```

And class selectors:

```
.a-box {  
  . . .  
}
```

However, selectors are an **extremely powerful** tool, and it's worth looking at a couple more cases!



CSS selectors

There are 3 main things that we can select:

- Tags
- Classes
- IDs

```
p {  
  ...  
}
```

```
.class {  
  ...  
}
```

```
#id {  
  ...  
}
```



CSS selectors

Tag selectors have **no prefix**

By selecting a tag, we are effectively selecting **every element** on the page with that **tag**

For example, this one would be styling every **paragraph** element

```
p {  
  ...  
}
```

```
.class {  
  ...  
}
```

```
#id {  
  ...  
}
```



CSS selectors

Class selectors are prefixed with a **dot**

As we've seen, this selector will target any element with a **matching class attribute**

```
p {  
  ...  
}
```

```
.class {  
  ...  
}
```

```
#id {  
  ...  
}
```



CSS selectors

ID selectors are prefixed with a hash

Similar to classes, they will select an element with the **matching id attribute**

A semantic distinction between **classes** and **ids** is that a **class** is expected to appear **multiple times** across elements, but an **id** should be **unique** on the page

```
p {  
  ...  
}
```

```
.class {  
  ...  
}
```

```
#id {  
  ...  
}
```



Multiple selectors

A cool feature of CSS is that it allows us to apply the same styling to **multiple selections**

To achieve this, all we have to do is **separate every selector** we want to apply styles to with a **comma**

```
body,  
.link,  
.section {  
padding: 10px;  
}
```



Multiple selectors

We can still add **more specific styling** to individual selections later

```
body,  
.link,  
.section {  
  padding: 10px;  
}
```

```
.link {  
  color: red;  
}
```



Multiple selectors

This allows us to **group similar stylings together**, instead of having to repeat ourselves

```
body {  
  padding: 10px;  
}
```

```
.section {  
  padding: 10px;  
}
```

```
.link {  
  padding: 10px;  
  color: red;  
}
```



Time for some live coding

Let's see if we can clean up our book's CSS!



Nested selectors

Another interesting feature of selectors is that we can **nest them**

Be careful with this syntax though, because these two similar examples **do completely different things!**

Q: What would you guess each of them does?

```
p .page #logo {  
  ...  
}
```

```
p.page#logo {  
  ...  
}
```



Nested selectors

The first one, **with spaces**, is saying:

Select elements with an **ID of #logo**, that are **inside** an element with a **class of .page**, that are **inside** a **paragraph** element

```
p .page #logo {  
  ...  
}
```

```
p.page#logo {  
  ...  
}
```



Nested selectors

These can be really useful when you want to select things that are **related**, for example:

```
.card .top { ... }
```

So if we had **multiple tops** for different things, here we make sure that we only **select the top of cards**, and **nothing else**

```
p .page #logo {  
  ...  
}
```

```
p.page#logo {  
  ...  
}
```



Nested selectors

Without spaces, it means:

Select elements with a **#logo** id, a **.page** class and a **p** tag

In other words, all elements that have **all 3 attributes combined!**

```
p .page #logo {
```

```
...
```

```
}
```

```
p.page#logo {
```

```
...
```

```
}
```



Specificity

Another good reason why we may want to use these kind of selectors is **specificity**

Take a look at these examples on the right

Q: Which color will each `<p>` be? Why?

```
p {  
  color: red;  
}  
  
p.content {  
  color: green;  
}  
  
p#logo {  
  color: blue;  
}
```

```
<p class="content" id="logo">Hi!</p>  
<p class="content">Hi!</p>  
<p>Hi!</p>
```



Specificity

When **multiple selectors** apply to the same element, **the more specific wins**

As a rule of thumb: **IDs > Classes > Tags**

IDs are worth **100 points** each, **classes 10**, and **tags** just **1 point**

Since you can have **multiple** of each **per selector**, they all **add up**, and the selector with the **highest score takes over**

You can read more about it [here](#)

And here's a [specificity calculator](#) 📊

```
p {  
  color: red;  
}  
p.content {  
  color: green;  
}  
p#logo {  
  color: blue;  
}
```

```
<p class="content" id="logo">Hi!</p>  
<p class="content">Hi!</p>  
<p>Hi!</p>
```



Quiz time!

Without looking it up, what score will each of these selectors get?

1. `#button.highlight`
2. `.page p b`
3. `main section`



Answer time!

Without looking it up, what score will each of these selectors get?

1. 110

2. 12

3. 2



Bonus round!

What is each selector selecting?

1. `#button.highlight`
2. `.page p b`
3. `main section`



Bonus round!

What is each selector selecting?

1. Elements with both an id of **#logo** and a class of **.highlight**
2. **bs** inside **ps** inside **.pages**
3. **sections** inside **main**